+ CAREVOICE HEALTH

BEGINNER PROJECT MANUAL · HINGLISH

CareVoice Project Guide

Project ke folders, front end, backend APIs, PostgreSQL database, Docker, local AI voice assistant, billing, testing aur future roadmap ko simple steps mein samjhein.

> **Is guide ko kaise use karein?**
> Pehle “Quick start” follow karein. Uske baad flow diagram aur file map dekhein. Jab feature change karna ho, relevant module aur uska test dhoondhne ke liye API aur roadmap sections use karein.

Prepared from the current CareVoice codebase · 29 September 2026
Local development guide · Windows PowerShell + Docker Compose

1. Project ko ek nazar mein

CareVoice ek learning/development patient-portal application hai. Patient account bana sakta hai, login kar sakta hai, apna profile dekh/sambhal sakta hai, text ya microphone se local AI assistant se baat kar sakta hai, aur clinic ke issue kiye invoices dekh sakta hai. Clinic staff account invoice bana sakta hai; Razorpay keys configure hon to patient online payment kar sakta hai.

Browser (React) → FastAPI REST API → PostgreSQL
↘ Ollama local AI
↘ Razorpay payment provider

Current working features

- Register/login, Argon2 password hash aur JWT bearer token.
- Account ke owner tak limited patient-profile CRUD.
- Browser voice-to-text, text chat aur spoken replies.
- Docker mein local Ollama `llama3.2:1b` (API key ke bina).
- Staff-only invoices, Razorpay checkout, signature verification aur webhook.
- Alembic database migrations, health endpoint, automated tests.

Abhi connected nahin

- Clinic appointment-booking integration.
- Hospital ke medical records/lab systems integration.
- Razorpay online payment ke liye clinic ke apne test/live credentials.
- Assistant diagnosis, treatment ya emergency service nahin hai.

> **Safety:** Yeh learning/development scaffold hai; HIPAA/DPDP certified ya clinically validated product nahin. Real patient data, real health records ya card details use na karein. Hamesha synthetic/demo data use karein. Emergency mein local emergency services/qualified clinician se sampark karein.

Table of contents

1. Quick start, Docker aur service ports
2. Folder/code structure aur frontend-to-backend flow
3. Backend layering, auth aur patient profile
4. PostgreSQL connection, tables aur migrations
5. API endpoints aur request examples
6. Voice, local AI aur Razorpay billing
7. Kaise test/debug karein
8. Abhi ki boundaries, improvement roadmap aur glossary

2. App chalana: beginner quick start

Commands ko project ke main folder mein PowerShell se run karein:

```
cd S:\project\ai-healthcare-voice-assistant
docker compose up -d
docker compose ps
```

Browser mein http://localhost:3000 kholein. Naya patient account “Create account” se banayein. API health check http://localhost:8000/api/v1/health; interactive API explorer http://localhost:8000/docs.

AI model pehli baar download karna

Ollama `docker compose up` ke saath start hota hai. Model ko ek baar download karein; yeh lagbhag 1.3 GB hai, isliye internet speed ke hisaab se waqt lag sakta hai:

```
docker compose exec ollama ollama pull llama3.2:1b
docker compose exec ollama ollama list
```

Model `ollama_data` Docker volume mein persist hota hai. Iske baad app ke Voice Assistant page par type ya voice se sawal karein. Browser mic permission maang sakta hai; voice recognition browser support/service par depend karti hai.

Useful Docker commands

```
docker compose ps
docker compose logs -f backend
docker compose logs -f ollama
docker compose logs -f frontend
docker compose restart backend
docker compose down
```

> **Data delete karne mein savdhani:** `docker compose down` data ko rakhta hai. `docker compose down -v` PostgreSQL aur AI model volumes hata deta hai; isse local database/model permanently delete ho sakte hain. Soch-samajh kar hi chalayein.

Kaunse ports kis kaam ke?

Port	Service	Use
3000	React/Vite frontend	Browser mein CareVoice UI.
8000	FastAPI backend	API aur Swagger docs.
5432	PostgreSQL	App database; sirf local machine par mapped.
11434	Ollama	Local AI server; sirf local machine par mapped.

Compose ports loopback (127.0.0.1) par bind hain, isliye normal setup mein LAN/public internet par expose nahin hote.

3. Code structure: files kahan hain?

```
ai-healthcare-voice-assistant/
|-- docker-compose.yml          # DB, API, UI, Ollama containers
|-- .env                        # Local secrets/config (git mein commit nahin)
|-- .env.example                # Config template
|-- README.md                   # Short setup guide
|-- docs/
|   |-- ARCHITECTURE.md         # Architecture notes
|   |-- PROJECT_GUIDE.html      # Is PDF ka printable source
|   `-- PROJECT_GUIDE.pdf       # Hinglish project manual
|-- backend/
|   |-- Dockerfile              # Python/FastAPI container aur migrations
|   |-- requirements.txt        # Python libraries
|   |-- alembic.ini             # Migration setup
|   |-- migrations/             # Database schema versions
|   `-- app/
|       |-- main.py             # FastAPI app; routes/health register
|       |-- cli.py              # Clinic staff account create command
|       |-- auth/security.py    # Password hash, JWT, signed-in user
|       |-- core/               # Settings, DB engine/session, exceptions
|       |-- models/             # SQLAlchemy database tables
|       |-- schemas/            # Input/output validation
|       |-- routes/             # HTTP/API endpoints
|       |-- controllers/        # Patient route coordination
|       |-- services/           # Patient rules and validations
|       `-- repositories/       # SQL/database operations
`-- frontend/
    |-- Dockerfile
    |-- package.json
    `-- src/
        |-- main.jsx            # React entry point
        |-- App.jsx             # Sign-in, navigation, overview, profile
        |-- App.css, portal.css  # Visual styling
        |-- services/api.js     # Shared HTTP requests/JWT header
        `-- components/
            |-- VoiceAssistant.jsx
            `-- Billing.jsx
```

Layer	Simple meaning	Example
Route	API ka darwaza: URL, login check, request/response.	`routes/patient_routes.py`
Schema	Incoming data sahi type/format mein hai ya nahin.	`schemas/patient.py`
Service	Business rule, jaise duplicate profile rokhna.	`services/patient_service.py`
Repository	SQL query aur row save/read/update/delete.	`repositories/patient_repository.py`
Model	Database table ki Python representation.	`models/patient.py`

Yeh “modular monolith” hai: ek backend deploy hota hai, par features alag modules mein rakhe hain. Naya feature ho to isi layering ko follow karna easy rahega.

4. Ek request ka end-to-end flow

1. User browser mein form bharta hai; React state value sambhalti hai.
2. Frontend ka `apiRequest()` JSON HTTP request `http://localhost:8000/api/v1` par bhejta hai.
3. Login ke baad saved JWT request ke `Authorization: Bearer ...` header mein jata hai.
4. FastAPI route request schema validate karta hai aur token se current user identify karta hai.
5. Patient route controller → service → repository ko call karta hai.
6. Repository SQLAlchemy ke zariye PostgreSQL se row read/write karta hai.
7. Backend JSON response deta hai; React UI success/error state aur screen update karti hai.

```
React component
    └── frontend/src/services/api.js
          └── HTTP + JSON + Bearer token
                └── FastAPI route (backend/app/routes/)
                      ├── authentication / schema validation
                      ├── controller → service → repository (patient)
                      └── SQLAlchemy session → PostgreSQL
```

Registration/login flow

- `POST /auth/register` password ka Argon2 hash banata hai; plain password database mein nahin rakhta.
- `POST /auth/login` email aur password verify karke signed JWT deta hai.
- Browser token ko `localStorage` mein rakhta aur protected API calls mein attach karta hai.
- App startup par `/auth/me` token validate karta hai. Token expired/invalid ho to sign out kar deta hai.
- JWT lifetime `ACCESS_TOKEN_MINUTES` se configure hoti hai (default 20 minute).

> **LocalStorage note:** Yeh dev app ka simple token storage hai. Production mein XSS defenses, short session strategy, token refresh/revocation aur MFA jaise design decisions/security review chahiye.

5. Backend aur API interaction

`backend/app/main.py` FastAPI application create karta hai, CORS allowlist set karta hai, routers register karta hai aur database health check deta hai. Application code mein route prefixes `/api/v1` hain.

Method + path	Login/role	Kaam
`POST /api/v1/auth/register`	Public	Naya patient account.
`POST /api/v1/auth/login`	Public	Login; JWT token response.
`GET /api/v1/auth/me`	JWT	Current user info.
`GET /api/v1/patients/`	JWT, owner	Current account ke profiles.
`POST /api/v1/patients/`	JWT, owner	Profile create.
`GET/PATCH/DELETE /api/v1/patients/{id}`	JWT, owner	Apna profile read/update/delete.
`POST /api/v1/assistant/chat`	JWT	Conversation local/hosted LLM ko bheje.
`GET /api/v1/billing/invoices`	JWT; staff sab dekh sakte hain	Invoice list.
`POST /api/v1/billing/invoices`	Clinic staff	Patient ke liye invoice issue.
`POST /api/v1/billing/checkout`	Invoice owner	Razorpay order create.
`POST /api/v1/billing/verify`	Invoice owner	Signature + captured payment verify.
`POST /api/v1/billing/webhooks/razorpay`	Webhook signature	Payment-captured event process.

Browser dev tools se request samajhna

1. Chrome/Edge mein **F12** → **Network** kholein.
2. Portal par action karein, jaise “Send message”.
3. `localhost:8000` wali request select karke status, URL, Request Payload aur Response dekhein.
4. `401` = login/token; `403` = role access; `404` = record not found; `422` = input validation; `503` = provider/config unavailable; `502` = external provider error.

6. PostgreSQL: database ka connection

PostgreSQL alag Docker container `db` mein chalta hai. Backend SQLAlchemy driver ke zariye is address par connect karta hai:

```
postgresql+psycopg://<user>:<password>@db:5432/<database>
```

Container ke andar hostname `db` Docker network ka service-name hai. Host Windows se app ko localhost ke port 5432 par publish kiya gaya hai. Backend setting `DATABASE_URL` hoti hai; production/secrets ko source code mein nahin likhna chahiye.

Tables ka basic map

Table	Kya rakhti hai	Important link
`users`	Full name, unique email, password hash, role, active flag, created time.	User ID har account ka key.
`patients`	Patient profile fields: naam, email, age, gender, created time.	`owner_user_id → users.id`.
`invoices`	Invoice number, description, amount in paise, currency, status, dates.	`owner_user_id → users.id`.
`payments`	Razorpay order/payment ID aur payment status.	`invoice_id → invoices.id`.
`alembic_version`	Kaunsa migration database par apply hua.	Migration revision tracking.

Database ko read-only tareeke se dekhna

```
docker compose exec db sh -c 'psql -U "$POSTGRES_USER" -d "$POSTGRES_DB"'
\dt
\d patients
SELECT id, full_name, age, gender, owner_user_id FROM patients LIMIT 10;
\q
```

Ye commands container ke andar `psql` kholti hain. Profile test karte waqt sirf synthetic data dekhein; database rows ko manually update/delete na karein.

Migrations: schema safely badalna

`backend/migrations/versions/0001_portal_billing.py` first migration hai: existing user/patient tables ko sambhalte hue patient ownership aur billing tables add karta hai. Backend startup mein `alembic upgrade head` chalne ke baad API start hoti hai. SQLAlchemy model file badal dena akela kaafi nahin—har schema change ke liye migration banana, review karna aur test karna zaroori hai.

> **Reset se pehle sochiye:** Database Docker volume mein persisted hai. `down -v` test data bhi mita dega. Migration mein existing data ko preserve/transform karne ka plan zaroor ho.

7. Safe API examples: PowerShell

API docs /docs se bhi endpoints manually try kar sakte hain. Neeche patient data ke liye dummy example diya hai:

1) Register and save token

```
$body = @{
  full_name = "Demo Patient"
  email = "demo.patient@example.test"
  password = "DemoPassword!123"
} | ConvertTo-Json

$auth = Invoke-RestMethod `
  -Uri http://localhost:8000/api/v1/auth/register `
  -Method Post -ContentType "application/json" -Body $body
$token = $auth.access_token
```

Agar account pehle se hai to `/auth/login` use karein. Real password file/chat mein paste na karein.

2) Apna account aur profiles dekhein

```
$headers = @{ Authorization = "Bearer $token" }
Invoke-RestMethod -Uri http://localhost:8000/api/v1/auth/me -Headers $headers
Invoke-RestMethod -Uri http://localhost:8000/api/v1/patients/ -Headers $headers
```

3) Dummy patient profile create karein

```
$profile = @{
  full_name = "Demo Patient"
  email = "demo.profile@example.test"
  age = 30
  gender = "prefer_not_to_say"
} | ConvertTo-Json

Invoke-RestMethod `
  -Uri http://localhost:8000/api/v1/patients/ `
  -Method Post -Headers $headers `
  -ContentType "application/json" -Body $profile
```

Important HTTP response codes

- `200` successful read/update; `201` created; `204` deleted.
- `401` token missing/expired; dobara login karke fresh token use karein.
- `403` aapke role ko action ki permission nahin.
- `404` record missing ya kisi aur account ka (ownership hide karne ke liye same response).
- `409` duplicate email / invoice already-paid conflict.
- `422` request body/age/email/input rules galat.
- `502/503` AI/payment provider error ya unavailable local service.

8. Frontend/UI ko kaise samjhein?

- `main.jsx` HTML root element mein React app mount karta hai.
- `App.jsx` sign-in/register screen, nav/sidebar, overview, health profile aur settings switch karta hai.
- `services/api.js` backend URL par fetch call karta, bearer token jodta, JSON parse karta aur errors ko readable message banata hai.
- `components/VoiceAssistant.jsx` chat messages, busy/error states, microphone transcript aur browser text-to-speech handle karta hai.
- `components/Billing.jsx` invoice list, staff invoice form, Razorpay checkout popup aur verification request handle karta hai.
- `App.css` base/dashboard styles; `portal.css` chat, invoice aur responsive additions.
- Vite dev server container port 5173 par chalta hai, host par port 3000 se khulta hai.

Voice assistant ka actual flow

1. Browser speech recognition microphone audio ko text transcript mein badalti hai; unsupported browser mein type karein.
2. Frontend last conversation turns ko `POST /api/v1/assistant/chat` par bhejta hai.
3. FastAPI bearer token verify karta aur system safety instructions add karta hai.
4. Backend internal Docker address `http://ollama:11434/v1/chat/completions` par model ko request bhejta hai.
5. Model reply browser mein dikhayi jaati hai; “Read aloud” browser speech synthesis chalata hai.

> **Privacy:** Chat CareVoice PostgreSQL mein save nahin hoti. Local model prompts Docker ke local Ollama ko jaate hain. Browser speech-recognition implementation browser/vendor ke speech service use kar sakti hai; sensitive identifiers share na karein.

UI mein error aaye to

1. Page refresh karein: **Ctrl + F5** purani screen/error clear karne mein madad karega.
2. `docker compose ps` mein `ollama` aur `backend` healthy hon.
3. `docker compose exec ollama ollama list` mein `llama3.2:1b` listed ho.
4. Logs ke liye `docker compose logs --tail 50 ollama` / `backend`.
5. Model missing ho to `docker compose exec ollama ollama pull llama3.2:1b`.

9. Billing aur Razorpay ka flow

1. Staff CLI se alag clinic-staff account banayein.
2. Staff Billing page se patient account ID, description aur rupee amount daal kar invoice issue karta hai.
3. Backend amount ko integer paise mein store karta (udaharan: ₹125.00 = 12500 paise).
4. Patient apni invoice list dekhkar “Pay now” karta. Browser payment amount decide nahin karta.
5. Backend invoice-owner verify karke Razorpay order banata; Checkout popup Razorpay mein payment leta hai.
6. Backend Razorpay signature verify karta aur provider se captured status/amount/currency confirm karta; tab invoice “paid” hoti hai.
7. Optional webhook captured payment ko confirm kar sakta hai; raw payload ka HMAC signature verify hota hai.

Staff account banana

```
docker compose exec backend python -m app.cli create-staff clinic@example.com
```

CLI password securely prompt karta hai; hash store hota hai. Public registration hamesha patient role banata hai—user khud ko staff nahin bana sakta.

Razorpay ke bina kya chalega?

Invoice create/list local app/database mein kaam kar sakte hain, lekin online payment ke liye Razorpay test credentials `.env` mein chahiye. Test keys se checkout end-to-end try karein. Real/live keys aur real charge se pehle legal, payment settlement/reconciliation aur security review zaroori hai.

Patient account ID portal ke Settings page par dikhta hai. Staff us ID se invoice patient ko assign karta hai. Unpaid invoice sirf uske owner patient ko dikhti hai.

10. Tests chalana aur debugging

Automated backend tests

```
docker compose exec backend pytest -q
docker compose exec backend python -m compileall -q app migrations tests
```

Current tests schema validation, account ownership/unauthorized access, no-key assistant configuration, local chat routing, staff-only invoice creation, invoice privacy aur mocked Razorpay captured-payment flow check karte hain. Real external Razorpay charge nahin karte.

Frontend production bundle

```
docker compose exec frontend npm run build
```

Service health

```
docker compose ps
Invoke-RestMethod http://localhost:8000/api/v1/health
Invoke-RestMethod http://localhost:11434/api/tags
docker compose exec ollama ollama list
```

PostgreSQL migration version

```
docker compose exec backend alembic current
```

Common problems

Symptom	Check / fix
Page load nahin	`docker compose ps`; frontend port 3000 aur backend port 8000 check.
Login 401	Email/password; expired token ho to sign out/sign in.
AI unavailable	Ollama health/model, backend logs aur `docker compose exec ollama ollama list`.
First AI response slow	Model pehli baar load ho raha ho sakta hai; local CPU inference ko waqt dein.
Payment not enabled	Razorpay TEST key/secret set karke backend restart karein.
DB/migration error	DB healthy? backend logs? `alembic current`. Volume delete karne se pehle backup/check.
Browser mic blocked	Site permission mein microphone allow karein; varna text chat use karein.

11. Naya feature aasani se kaise add karein?

Example: appointments feature add karte waqt ek hi file mein sab kuch daalne ke bajaye yeh sequence follow karein:

1. Requirement define karein: appointment fields, roles, permissions aur user flow.
2. `models/appointment.py` mein SQLAlchemy table/entity.
3. `schemas/appointment.py` mein create/update/read validation.
4. `migrations/versions/` mein reviewed Alembic migration.
5. `repositories/appointment_repository.py` mein database queries.
6. `services/appointment_service.py` mein availability/business rules.
7. `routes/appointment_routes.py` mein authentication aur HTTP endpoints.
8. `main.py` mein router register karein; auth/ownership enforce karein.
9. `frontend/src/components/` mein UI; existing `apiRequest()` reuse karein.
10. Unit/API tests likhein: allowed user, disallowed user, invalid input, success case.
11. Build/tests pass karke README/API docs update karein.

Is architecture se maintenance kaise easy hoti hai?

- Routes ke andar complex SQL/rules na bharne se business logic testable hoti hai.
- Shared frontend API helper se har component ka token/error handling consistent.
- Models + migrations se database changes repeatable aur versioned.
- Modular monolith deployment ko simple rakhta aur feature files alag rakhta hai.
- Clear unconnected-state fake appointment/medical data se bachata hai.

12. Future roadmap: kya improve/add karein?

Priority	Feature / improvement	Kyun useful hai?
1 · Security	Rate limiting, audit logging, refresh/logout token revoke, MFA, staff admin workflow, strict role checks.	Accounts aur sensitive data ko stronger protection.
1 · Tests	Frontend component/e2e tests; DB migration test; webhook replay/idempotency tests; CI pipeline.	Change ke baad regression jaldi pakadna.
1 · Operations	Production secrets manager, HTTPS reverse proxy, DB backup + restore exercise, logs/metrics/alerts.	Deployment aur recovery reliable banaye.
2 · Scheduling	Appointment model, clinic calendar, provider availability, time zone, cancel/reschedule rules, notifications.	"Appointments" UI ko real service banana.
2 · Records	Consent, secure file storage, access rules, malware scan, audit, clinician/provider integrations.	Medical records ko safe sharing workflow dena.
2 · Billing	Invoice PDF/email, refunds, failed payment state, webhook idempotency, reconciliation, receipts/tax.	Production payment operations complete karna.
2 · Accessibility	Keyboard/screen reader audit, language selection, speech locale control, mobile UX tests.	Zyada users asani se use kar sakein.
3 · AI	Model/provider setting UI, timeouts/retry/streaming, feedback, output evaluation, safe health-content review.	Helpful but controlled AI experience.
3 · Maintainability	Typed frontend, lint/format standards, generated API client, structured logs, dependency scanning.	Code changes consistent aur errors trackable.

Real healthcare production se pehle: compliance/privacy/legal review, qualified clinical governance, threat modelling/security testing, data retention/deletion rules, breach response aur certified infrastructure ki zaroorat ho sakti hai. Sirf code add kar dena compliance certification nahin hota.

13. Chhota glossary

Term	Asaan matlab
React	Browser UI ko components mein banane ki JavaScript library.
Vite	Frontend dev server aur production bundle tool.
FastAPI	Python framework jo HTTP API endpoints expose karta hai.
REST API	HTTP URL/method/JSON se frontend-backend communication.
PostgreSQL	Relational database; data tables/rows mein store hota hai.
SQLAlchemy	Python code aur SQL database ke beech ORM/toolkit.
Migration / Alembic	Database schema change ko versioned script se apply karna.
JWT / Bearer token	Signed temporary login credential jo protected API ko bheja jata hai.
Docker Compose	Multiple app containers ko ek command se run karna.
Ollama / LLM	Ollama local AI model server; LLM text conversation generate karta hai.
Webhook	Razorpay jaise provider ka server-to-server event notification.
CI	Code push hone par tests/build automatically chalane ka process.

Sabse pehla practice task

App kholein → patient account register karein → "My Health" profile save karein → API docs mein `GET /api/v1/health` try karein → AI model ke saath ek general, non-sensitive sawal poochhein → `docker compose exec backend pytest -q` chalayein. Isse UI, API, database, AI aur tests ka basic flow samajh aayega.

Project setup ka short reference: repository ke root mein `README.md`; design summary ke liye `docs/ARCHITECTURE.md`; iss guide ka source `docs/PROJECT_GUIDE.html`.

CareVoice is a development/educational project. Is guide mein diye gaye commands local demo environment ke liye hain; real patient/payment data par use karne se pehle qualified engineering, security, clinical aur privacy review zaroor lein.